

TTB Alcohol Label Verifier

Design, Implementation, and Test Environment

Todd Baker

2026-06-24

Contents

1	Project Overview	2
1.1	Problem Statement	2
1.2	Regulatory Scope	2
2	Architecture	2
2.1	Technology Stack	2
2.2	Processing Pipeline	3
3	Probabilistic Models	3
3.1	Design Rationale: Speed as the Primary Constraint	3
3.2	CRF Sequence Tagger (Primary Field Extractor)	3
3.3	Hidden Markov Model (Structural Anomaly Detector)	4
3.4	Brand Markov Chain (Plausibility Scorer)	5
3.5	Label Region Detection: Prototype vs. Production	5
3.6	Class-Type-Aware ABV Validation	5
3.7	COLA Lookup Store	5
4	Test Dataset	6
4.1	COLA Registry Scrape	6
4.2	Synthetic Test Labels	6
5	File Layout	6
6	Environment Reproduction	7
6.1	Manual (Fedora 42)	7
6.2	Ansible (Automated)	7
7	Key Findings and Limitations	8

1 Project Overview

The TTB Alcohol Label Verifier is a prototype application that automates compliance checking of Alcohol and Tobacco Tax and Trade Bureau (TTB) Certificate of Label Approval (COLA) labels against their submitted application data.

1.1 Problem Statement

TTB reviewers currently spend 5–10 minutes per label performing manual field-by-field comparison between a physical or digital label image and the approved COLA application form. Key requirements from stakeholder interviews:

- Sub-5-second processing per label
- Fully offline (government firewall blocks outbound ML API endpoints)
- Simple UI suitable for non-technical reviewers
- Batch upload support for high-volume review sessions

1.2 Regulatory Scope

The verifier checks five mandatory label fields:

Field	Match Type	Regulatory Basis
Brand Name	Fuzzy ($\geq 90\%$)	27 CFR § 5.63
Class/Type	Fuzzy ($\geq 85\%$)	27 CFR § 5.35
ABV Percent	Float + range	27 CFR § 5.36, § 5.121
Net Contents	Fuzzy ($\geq 85\%$)	27 CFR § 5.38
Government Warning	Exact (normalized)	27 CFR § 16.21

2 Architecture

2.1 Technology Stack

Layer	Technology
Language	Go 1.22
Image preprocessing	GoCV (OpenCV bindings for Go)
OCR	Tesseract 4 via <code>exec.Command</code> (<code>--psm 4</code>)
Field extraction (primary)	Linear-Chain CRF (sklearn-crfsuite, Python)
Structural anomaly detection	Hidden Markov Model with Forward algorithm (Python)
Brand plausibility scoring	Character bigram Markov chain (Python)
Fuzzy matching	RapidFuzz Levenshtein distance
Web server	<code>net/http</code> (no framework)
Frontend	Vanilla HTML/CSS/JS

2.2 Processing Pipeline

```

Upload image
  |
  v
GoCV: MSER region detection -> label crop
      Grayscale -> Otsu binarize -> 2x cubic upscale
  |
  v
Tesseract OCR (--psm 4, raw text lines)
  |
  v
CRF sequence tagger (BIO tags):
  B-BRAND  I-BRAND  B-TYPE  I-TYPE
  B-ABV    B-NET    B-WARN  I-WARN  B-MFR  O
  |
  +---> HMM Forward pass  -> log-likelihood anomaly score
  |
  +---> Markov chain      -> brand plausibility score
  |
  v
Comparison engine:
  brand_name  -> Levenshtein fuzzy >= 90%
  class_type  -> Levenshtein fuzzy >= 85%
  abv_percent -> class-type-aware range + +/-0.1 tolerance
  net_contents -> Levenshtein fuzzy >= 85%
  gov_warning -> content >= 88% AND >= 70% uppercase (ALL CAPS check)
  |
  v
ComplianceResult {PASS/FAIL, per-field verdicts, anomaly scores}

```

3 Probabilistic Models

3.1 Design Rationale: Speed as the Primary Constraint

Stakeholder interviews established a hard latency ceiling: the prior scanning vendor pilot was abandoned because 30–40 seconds per label made manual review faster. When a human agent can check five labels in the time a machine handles one, the machine is not a productivity tool—it is a liability. This constraint shaped every model selection decision: no large generative model, no cloud API call, no on-demand model download. All inference runs locally on pre-trained parameters and completes in under one second per label.

3.2 CRF Sequence Tagger (Primary Field Extractor)

A **Linear-Chain Conditional Random Field** (CRF) is a discriminative probabilistic model that assigns a label sequence to an input sequence. Unlike a generative model such

as an HMM, the CRF models $P(\text{labels} \mid \text{observations})$ directly, allowing it to condition on overlapping, non-independent features simultaneously.

Training Corpus

The CRF was trained on 30 annotated synthetic labels generated from real brand names and class types drawn from the TTB COLA Public Registry (ttbonline.gov). Each line of OCR output was tagged with a BIO (Beginning–Inside–Outside) label:

BIO Tag	Meaning
B-BRAND / I-BRAND	Brand name (first / continuation line)
B-TYPE / I-TYPE	Class/type designation
B-ABV	Alcohol by volume line
B-NET	Net contents line
B-WARN / I-WARN	Government warning (first / continuation)
B-MFR / I-MFR	Manufacturer/bottler address
O	Non-field line

Features and Training

Each line is featurised with: `is_allcaps`, `has_percent`, `has_volume_unit`, `starts_gov_warning`, bourbon keyword presence, word count bucket, character count bucket, relative position on the label, and ± 1 context window features from adjacent lines. Weights are learned by L-BFGS optimisation (`sklearn-crfsuite`, $c_1 = 0.05$, $c_2 = 0.1$, 200 iterations). The trained model is serialised to `testdata/crf_model.pkl`. Benchmark accuracy on the 30-label corpus: **100%**.

3.3 Hidden Markov Model (Structural Anomaly Detector)

A **Hidden Markov Model** (HMM) is a generative probabilistic model of a sequence of observations emitted by an unobserved (hidden) state sequence. Two algorithms are used:

Viterbi decoder Finds the single most likely hidden state sequence given the observations—used as a second field-extraction path for benchmarking against the CRF.

Forward algorithm Sums over *all* possible state paths to compute $\log P(\text{token sequence} \mid \text{HMM})$ —used as a structural anomaly score. A score significantly below the mean for known-good labels (< -4.0 per token) indicates that the label’s token ordering does not match any bourbon label structure. This catches non-label images, foreign labels, and fraudulent mock-ups without requiring a separate classifier.

The HMM states correspond to the same field types as the CRF BIO tags; transition and emission parameters were hand-specified from domain knowledge of bourbon label layout conventions and then refined against the 30-label corpus.

3.4 Brand Markov Chain (Plausibility Scorer)

A character **bigram Markov chain** is trained on the corpus of 609 unique brand names extracted from the TTB COLA Public Registry scrape (1,004 whisky records, dates 2020–2026). The model learns the transition probability $P(c_n | c_{n-1})$ for every character pair that appears in a legitimately registered bourbon brand name.

At inference time, the extracted brand name is scored against the transition matrix. The log-probability per character is returned as `markov_score` in the API response:

- Scores near 0: consistent with real bourbon naming conventions.
- Score < -3.5 : flagged **SUSPICIOUS**—indicates either OCR noise ("BUFFALO TR4CE") or a phonetically plausible counterfeit ("Elijah Crais").

The bigram corpus is serialised to `testdata/brand_markov.json` and loaded at server startup with no runtime dependency on the training script.

3.5 Label Region Detection: Prototype vs. Production

The prototype uses **GoCV MSER** (Maximally Stable Extremal Regions), a purely geometric algorithm built into OpenCV, to locate text regions. MSER requires no trained model weights and keeps the binary fully offline.

The planned production approach is **MediaPipe TextDetector**, a lightweight learned model that returns individual text bounding boxes. Each bounding box would be fed to Tesseract independently—a meaningfully better approach because each OCR call sees a single isolated text region rather than the full label, reducing noise from decorative elements and background texture. MediaPipe was chosen over YOLOv8 (the original candidate) because YOLO is a general object detector that would locate the bottle rather than the text fields directly. The CRF tagger and compliance engine are detector-agnostic; replacing MSER with MediaPipe requires only a change to `internal/preprocess/preprocess.go`.

3.6 Class-Type-Aware ABV Validation

Standard ABV tolerance of $\pm 0.1\%$ is insufficient for all whisky subtypes. The comparator applies 27 CFR regulatory ranges before the form-value check:

Class	Type	Code	Description	ABV Rule
111			BOURBON WHISKY BIB	Exactly 50.0% (27 CFR § 5.121)
101			STRAIGHT BOURBON WHISKY	40.0–80.0% (27 CFR § 5.36)
141			BOURBON WHISKY	40.0–80.0% (27 CFR § 5.36)

3.7 COLA Lookup Store

The server pre-loads `testdata/labels.csv` at startup into an in-memory map keyed by normalized brand name. During single-label and batch verification, missing expected field

values are auto-populated from the store. The `POST /import` endpoint accepts a replacement CSV and hot-reloads the store without a server restart.

The `GET /api/lookup?brand=MAKER%27S+MARK` endpoint supports JavaScript autocomplete in the single-label form.

4 Test Dataset

4.1 COLA Registry Scrape

Real COLA metadata was obtained from the TTB COLA Public Registry (ttbonline.gov) using a custom `webdriver-tools` Selenium toolkit (`~/vscode/webdriver-tools/`).

- Search: class type range 100–199 (all whisky), dates 2020–2026-06-24
- Total records: 1,004 (registry export capped at 1,000)
- Bourbon records: 990 (class type 101, STRAIGHT BOURBON WHISKY)
- Unique brand names: 609
- Top origin states: Kentucky (123), Tennessee (63), California (60)

Limitation: Label images are not available on the public COLA registry (login required). The `scripts/cola_scraper.py` correctly populates the metadata CSV; image retrieval requires authenticated TTBOonline access.

4.2 Synthetic Test Labels

30 synthetic label images were generated via Pillow (`scripts/build_testdata.py`) using real brand names and class types from the COLA registry scrape:

Category	Count
PASS (all fields correct)	20
FAIL – government warning (title case)	5
FAIL – ABV mismatch (+5%)	5
Total	30

Images are stored in `testdata/images/`; ground-truth CSV in `testdata/labels.csv`.

5 File Layout

```
~/vscode/ttb-label-verifier-go/
cmd/server/main.go      HTTP server, /verify /batch /import /api/lookup
internal/
  agent/
    agent.go             Ollama vision agent (moondream -> llava:7b)
```

tesseract.go	Gosseract OCR fallback
compare/compare.go	Field comparator (fuzzy + ABV range + warning)
models/models.go	LabelFields, FieldVerdict, ComplianceResult
preprocess/preprocess.go	GoCV MSER -> Otsu -> denoise -> upscale
store/store.go	In-memory COLA lookup store (CSV-backed)
web/	
templates/index.html	Two-tab UI (Single Label / Batch Review)
static/style.css	Dark header, color-coded verdicts
static/app.js	fetch-based verify/batch/lookup/download
prompts/system.txt	TTB ruleset + 3 few-shot examples
testdata/	
labels.csv	30 synthetic records (brand, class, abv, net)
images/cola_NNNN.jpg	Synthetic label images
scripts/	
cola_scraper.py	WebDriver scraper for TTBOonline COLA registry
build_testdata.py	Synthetic image + CSV generator
go.mod	

6 Environment Reproduction

6.1 Manual (Fedora 42)

```
# System dependencies
sudo dnf install opencv opencv-devel tesseract tesseract-devel golang git

# Vision agent
curl -fsSL https://ollama.com/install.sh | sudo sh
ollama pull moondream      # 1.8 GB
ollama pull llava:7b       # 4.7 GB (optional fallback)

# Build and run
cd ~/vscode/ttb-label-verifier-go
go mod tidy
go run ./cmd/server

# Generate test dataset
python3 scripts/build_testdata.py
```

6.2 Ansible (Automated)

The `ttb_label_verifier` role in `~/vscode/ansible/roles/` handles full environment setup including system packages, Ollama install, model pulls, source sync, binary build, and systemd service registration.

```
ansible-playbook site.yml --tags ttb
```

Key role defaults (defaults/main.yml):

- `ttb_install_dir`: /opt/ttb-label-verifier
- `ttb_port`: 8080
- `ttb_ollama_models`: moondream, llava:7b
- `ttb_colo_csv`: testdata/labels.csv

7 Key Findings and Limitations

Public COLA registry does not expose label images. Structured metadata (brand, class type, dates, origin state) is publicly accessible. The actual label image files require authenticated TTBOonline access. Synthetic images are sufficient for pipeline validation; production deployment would require access to TTB's internal COLA document store.

Class type codes encode regulatory constraints. The 3-digit numeric class type codes carry statutory ABV requirements. Bottled-in-Bond products (code 111) must read exactly 50.0%; this is now enforced independently of the submitted form value.

COLA store enables sub-second auto-fill. With 1,000+ records in the in-memory map, expected-value lookup by brand name is $O(1)$ and adds < 1 ms overhead per request. Hot-reload via `/import` means the CSV can be refreshed from a new TTB bulk export without a restart.

Government warning is the highest-confidence check. The exact warning text is fixed by statute (27 CFR § 16.21). Case normalization catches the most common failure mode (title case instead of all-caps bold).